

Boolean Algebra

John Winans

January 3, 2023

1 Basic Operations

We describe Boolean values as either *false* or *true*.¹

In a system that represents information numerically using only binary digits:

- $0 = \text{false}$
- $1 = \text{true}$

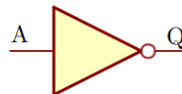
The following three basic Boolean operations represent the only operators we will use when reducing equations into their simplest form.

1.1 NOT

A function whose output is the opposite of its input.

$$Q = \overline{A} \tag{1}$$

A	Q
0	1
1	0



Depending on the context, any of the following forms might be seen used to express the **NOT** function applied to the variable a :

- $\neg a$
- $\bar{a}$
- a'
- $\sim a$ (C bitwise operator)
- $!a$ (C logical operator)

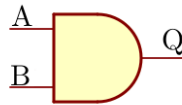
¹In non-binary systems it might help to consider a more general definition where zero is considered *false* and anything that is not *false* is *true*.

1.2 AND

A function whose output is *true* if, and only if, all of its inputs are *true*.

$$Q = A \wedge B \quad (2)$$

A	B	Q
0	0	0
0	1	0
1	0	0
1	1	1



Depending on the context, any of the following forms might be seen used to express the **AND** function applied to the variables *a* and *b*

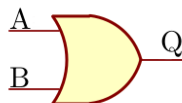
- $a \wedge b$
- $a \cdot b$
- ab
- $a \ \& \ b$ (C bitwise operator)
- $a \ \&\& \ b$ (C logical operator)

1.3 OR

A function whose output is *true* if any one, or both, of its inputs is/are *true*.

$$Q = A \vee B \quad (3)$$

A	B	Q
0	0	0
0	1	1
1	0	1
1	1	1



Depending on the context, any of the following forms might be seen used to express the **OR** function applied to the variables *a* and *b*

- $a \vee b$

- $a + b$
- $a \mid b$ (C bitwise operator)
- $a \parallel b$ (C logical operator)

2 Composing New Functions

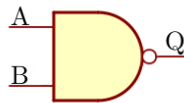
We can combine the above three functions to create new ones.

2.1 NAND

A function whose output is *false* if, and only if, all of its inputs are *true*.

$$Q = \overline{A \wedge B} \quad (4)$$

A	B	Q
0	0	1
0	1	1
1	0	1
1	1	0



Depending on the context, any of the following forms might be seen used to express the **NAND** function applied to the variables a and b

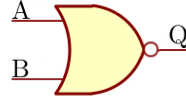
- $\overline{a \wedge b}$
- $\overline{ab}$
- $\overline{a \cdot b}$
- $\sim(a \ \& \ b)$ (C bitwise operators)
- $!(a \ \&\& \ b)$ (C logical operators)

2.2 NOR

A function whose output is *true* if, and only if, all of its inputs are *false*.

$$Q = \overline{A \vee B} \quad (5)$$

A	B	Q
0	0	1
0	1	0
1	0	0
1	1	0



Depending on the context, any of the following forms might be seen used to express the **NOR** function applied to the variables a and b

- $\overline{a \vee b}$
- $\overline{a + b}$
- $\sim(a \mid b)$ (C bitwise operators)
- $!(a \parallel b)$ (C logical operators)

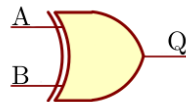
2.3 XOR (Exclusive OR)

A function whose output is *true* when an odd number of its inputs are *true*. We call this *odd parity*. In the special case (when there are only two inputs) the **XOR** function output is *true* when the two inputs are different and *false* when they are the same.

$$Q = A \oplus B \quad (6)$$

$$= \overline{A} \wedge B \vee A \wedge \overline{B} \quad (7)$$

A	B	Q
0	0	0
0	1	1
1	0	1
1	1	0



Depending on the context, any of the following forms might be seen used to express the **XOR** function applied to the variables a and b

- $a \oplus b$
- $a \wedge b$ (C bitwise operator)

3 Material Implication

A function with two inputs a and b whose output is *true* if either a is *false* or a is *true* when b is *true*.

$$Q = A \rightarrow B \quad (8)$$

$$= \overline{A} \vee B \quad (9)$$

A	B	Q
0	0	1
0	1	1
1	0	0
1	1	1

Depending on the context, any of the following forms might be seen used to express the **IMP** function applied to the variables a and b

- $a \rightarrow b$
- `a ? b : true` (C conditional/ternary operator)

4 All Possible Functions With Two Inputs

Consider the following:

- How many ways can we arrange 2 bits? (Answer: $2^2 = 4$)
- How many ways can we arrange 4 bits? (Answer: $2^4 = 16$)

Therefore:

1. There are exactly four possible ways to arrange two one-bit values.
2. There are exactly sixteen possible ways to arrange four one-bit values.

Conclusion: There are 16 possible Boolean functions that have two one-bit inputs a and b :

a	b	0	$a \wedge b$	$\overline{a \rightarrow b}$	a	$\overline{b \rightarrow a}$	b	$a \oplus b$	$a \vee b$	$\overline{a \vee b}$	$\overline{a \oplus b}$	$\overline{b}$	$b \rightarrow a$	$\overline{a}$	$a \rightarrow b$	$\overline{a \wedge b}$	1
0	0	0	0	0	0	0	0	0	0	1	1	1	1	1	1	1	1
0	1	0	0	0	0	1	1	1	1	0	0	0	0	1	1	1	1
1	0	0	0	1	1	0	0	1	1	0	0	1	1	0	0	1	1
1	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1
		0	1	2	3	4	5	6	7	8	9	a	b	c	d	e	f

5 DeMorgan's Morgan's Laws

https://en.m.wikipedia.org/wiki/De_Morgan's_laws

De Morgan observed the following relationships:

$$\overline{a \wedge b} = \overline{a} \vee \overline{b} \quad (10)$$

$$\overline{a \vee b} = \overline{a} \wedge \overline{b} \quad (11)$$

Proof by truth table:

a	b	$\overline{a}$	$\overline{b}$	$a \wedge b$	$\overline{a \wedge b}$	$\overline{a} \vee \overline{b}$	$a \vee b$	$\overline{a \vee b}$	$\overline{a} \wedge \overline{b}$
0	0	1	1	0	1	1	0	1	1
0	1	1	0	0	1	1	1	0	0
1	0	0	1	0	1	1	1	0	0
1	1	0	0	1	0	0	1	0	0

6 Completeness

Completeness refers to the fact that the *basic operations* provide a sufficient basis to describe all other possible Boolean operations.

https://en.wikipedia.org/wiki/Boolean_algebra#Completeness

A proof showing that NAND is *complete* by showing a truth table for a NOT, AND and OR function that are constructed only using the NAND function. ²

$$\bar{a} = \overline{a \wedge a} \quad (12)$$

$$a \wedge b = \overline{\overline{a \wedge b} \wedge \overline{a \wedge b}} \quad (13)$$

$$a \vee b = \overline{\overline{a \wedge a} \wedge \overline{b \wedge b}} \quad (14)$$

a	b	$\bar{a}$	$\overline{a \wedge a}$	$\overline{b \wedge b}$	$\overline{a \wedge a} \wedge \overline{b \wedge b}$	$\overline{\overline{a \wedge a} \wedge \overline{b \wedge b}}$	$a \vee b$	$\overline{a \wedge b}$	$\overline{\overline{a \wedge b} \wedge \overline{a \wedge b}}$	$a \wedge b$
0	0	1	1	1	1	0	0	1	0	0
0	1	1	1	0	0	1	1	1	0	0
1	0	0	0	1	0	1	1	1	0	0
1	1	0	0	0	0	1	1	0	1	1

7 Operator Precedence & Parenthesis

- The NOT operator has higher precedence than AND.
- The AND operator has higher precedence than OR.
- The OR operator has higher precedence than IMPLICATION.

Note that XOR is a function of one or more of these operators. Therefore its precedence is a matter of its implementation. Languages like C that include an explicit XOR operator put its precedence between that of the AND and OR operators.

Parenthesis can be used to manage the order of operations when the precedence rules would otherwise cause an undesirable result:

$$a \vee b \wedge c = a \vee (b \wedge c) \quad (15)$$

$$\neq (a \vee b) \wedge c \quad (16)$$

8 Laws of Boolean Algebra

Summarized from https://en.wikipedia.org/wiki/Boolean_algebra.

²This along with DeMorgan's laws and the above discussion on the composition of new functions demonstrates that the NAND function alone can be used to perform all 16 possible Boolean functions.

$a \wedge (b \wedge c) = (a \wedge b) \wedge c$	Associativity of $\wedge$	(17)
$a \vee (b \vee c) = (a \vee b) \vee c$	Associativity of $\vee$	(18)
$a \wedge b = b \wedge a$	Commutativity of $\wedge$	(19)
$a \vee b = b \vee a$	Commutativity of $\vee$	(20)
$a \wedge (b \vee c) = (a \wedge b) \vee (a \wedge c)$	Distributivity of $\wedge$ over $\vee$	(21)
$a \vee (b \wedge c) = (a \vee b) \wedge (a \vee c)$	Distributivity of $\vee$ over $\wedge$	(22)
$a \wedge 1 = a$	Identity for $\wedge$	(23)
$a \vee 0 = a$	Identity for $\vee$	(24)
$a \wedge 0 = 0$	Annihilator for $\wedge$	(25)
$a \vee 1 = 1$	Annihilator for $\vee$	(26)
$a \wedge a = a$	Idempotence $\wedge$	(27)
$a \vee a = a$	Idempotence $\vee$	(28)
$a \wedge (a \vee b) = a$	Absorption 1	(29)
$a \vee (a \wedge b) = a$	Absorption 2	(30)
$a \wedge \bar{a} = 0$	Complementation of $\wedge$	(31)
$a \vee \bar{a} = 1$	Complementation of $\vee$	(32)
$\bar{\bar{a}} = a$	Double negation	(33)
$\bar{a} \wedge \bar{b} = \overline{a \vee b}$	DeMorgan's 1	(34)
$\bar{a} \vee \bar{b} = \overline{a \wedge b}$	DeMorgan's 2	(35)

8.1 An Exercise

You should be able to show that each of the following equations are true by applying the above Boolean laws and by showing the truth tables for each case.

$$a \wedge b = \overline{\bar{a} \vee \bar{b}} \quad (36)$$

$$a \vee b = \overline{\bar{a} \wedge \bar{b}} \quad (37)$$

$$a \vee \bar{a} \wedge b = a \vee b \quad (38)$$

$$\bar{a} \vee a \wedge b = \bar{a} \vee b \quad (39)$$

$$(a \vee b) \wedge (c \vee d) = a \wedge c \vee a \wedge d \vee b \wedge c \vee b \wedge d \quad (40)$$

For example, a proof of Equation 39:

$$f = \bar{a} \vee a \wedge b \quad \text{Given} \quad (41)$$

$$= \bar{a} \vee (a \wedge b) \quad \text{Show operator precedence} \quad (42)$$

$$= (\bar{a} \vee a) \wedge (\bar{a} \vee b) \quad \text{Distributivity of } \vee \text{ over } \wedge \quad (43)$$

$$= (a \vee \bar{a}) \wedge (\bar{a} \vee b) \quad \text{Commutativity of } \vee \quad (44)$$

$$= 1 \wedge (\bar{a} \vee b) \quad \text{Complementation of } \vee \quad (45)$$

$$= (\bar{a} \vee b) \wedge 1 \quad \text{Commutativity of } \wedge \quad (46)$$

$$= \bar{a} \vee b \quad \text{Identity for } \wedge \quad (47)$$

9 Minterm (lower-case m) (SOP form)

See also: https://en.wikipedia.org/wiki/Canonical_normal_form

Any Boolean function can be expressed in SOP form.³

In a minterm, each variable may appear only once as a or $\bar{a}$. For a three-input function, there are $2^3 = 8$ minterms:

$$m0 = \bar{a} \wedge \bar{b} \wedge \bar{c} \quad (48)$$

$$m1 = \bar{a} \wedge \bar{b} \wedge c \quad (49)$$

$$m2 = \bar{a} \wedge b \wedge \bar{c} \quad (50)$$

$$m3 = \bar{a} \wedge b \wedge c \quad (51)$$

$$m4 = a \wedge \bar{b} \wedge \bar{c} \quad (52)$$

$$m5 = a \wedge \bar{b} \wedge c \quad (53)$$

$$m6 = a \wedge b \wedge \bar{c} \quad (54)$$

$$m7 = a \wedge b \wedge c \quad (55)$$

Note that the appearance of the NOT-bars match the pattern of zeros when counting from zero to seven in binary.

Use of minterms in an equation allows a shorter notation as shown below:

$$f = m1 \vee m2 \vee m4 \vee m7 \quad (56)$$

$$= (\bar{a} \wedge \bar{b} \wedge c) \vee (\bar{a} \wedge b \wedge \bar{c}) \vee (a \wedge \bar{b} \wedge \bar{c}) \vee (a \wedge b \wedge c) \quad (57)$$

Parenthesis added for readability.

The long form shown in equation 57 suggests a rationale for why it is called SOP (Sum Of Products.)

	a	b	c	$(\bar{a} \wedge \bar{b} \wedge c)$	$(\bar{a} \wedge b \wedge \bar{c})$	$(a \wedge \bar{b} \wedge \bar{c})$	$(a \wedge b \wedge c)$	f
$m0$	0	0	0	0	0	0	0	0
$m1$	0	0	1	1	0	0	0	1
$m2$	0	1	0	0	1	0	0	1
$m3$	0	1	1	0	0	0	0	0
$m4$	1	0	0	0	0	1	0	1
$m5$	1	0	1	0	0	0	0	0
$m6$	1	1	0	0	0	0	0	0
$m7$	1	1	1	0	0	0	1	1

As can be seen in the truth table, the “mechanics” driving the SOP equation is based on identifying those specific function input patterns where the output is *true*. By using an OR gate to drive the output, it will set its output *true* any time one of the recognized input terms is *true*.

10 Maxterm (upper-case M) (POS form)

Same idea as the minterm, but inverted and use the $\vee$ instead of the $\wedge$ function:

³See: Peter J. Pahl, Rudolf Damrath, “Mathematical Foundations of Computational Engineering: A Handbook,” 2001, Springer, page 15. <https://tinyurl.com/ujv8q71>

Any Boolean function can be expressed in POS form.

$$M0 = a \vee b \vee c \quad (58)$$

$$M1 = a \vee b \vee \bar{c} \quad (59)$$

$$M2 = a \vee \bar{b} \vee c \quad (60)$$

$$M3 = a \vee \bar{b} \vee \bar{c} \quad (61)$$

$$M4 = \bar{a} \vee b \vee c \quad (62)$$

$$M5 = \bar{a} \vee b \vee \bar{c} \quad (63)$$

$$M6 = \bar{a} \vee \bar{b} \vee c \quad (64)$$

$$M7 = \bar{a} \vee \bar{b} \vee \bar{c} \quad (65)$$

Note that the appearance of the NOT-bars match the pattern of ones when counting from zero to seven in binary.

$$f = M0 \wedge M1 \wedge M2 \wedge M4 \quad (66)$$

$$= (a \vee b \vee c) \wedge (a \vee b \vee \bar{c}) \wedge (a \vee \bar{b} \vee c) \wedge (\bar{a} \vee b \vee c) \quad (67)$$

The long form shown in equation 67 is often called a POS (Product Of Sums.)

	<i>a</i>	<i>b</i>	<i>c</i>	$(a \vee b \vee c)$	$(a \vee b \vee \bar{c})$	$(a \vee \bar{b} \vee c)$	$(\bar{a} \vee b \vee c)$	<i>f</i>
<i>M0</i>	0	0	0	0	1	1	1	0
<i>M1</i>	0	0	1	1	0	1	1	0
<i>M2</i>	0	1	0	1	1	0	1	0
<i>M3</i>	0	1	1	1	1	1	1	1
<i>M4</i>	1	0	0	1	1	1	0	0
<i>M5</i>	1	0	1	1	1	1	1	1
<i>M6</i>	1	1	0	1	1	1	1	1
<i>M7</i>	1	1	1	1	1	1	1	1

As can be seen in the truth table, the “mechanics” driving the POS equation is based on identifying those specific function input patterns where the output is *false*. By using an AND to drive the output, it will set its output *false* any time one of the recognized input terms is *false*.

10.1 Relationship Between Minterms and Maxterms

The complement of a maxterm, such as $\overline{M5}$, is the respective minterm *m5*.

This can be verified with DeMorgan’s as in:

$$M5 = \bar{a} \vee b \vee \bar{c} \quad \text{Given} \quad (68)$$

$$m5 = a \wedge \bar{b} \wedge c \quad \text{Given} \quad (69)$$

$$\overline{M5} = \overline{\bar{a} \vee b \vee \bar{c}} \quad \text{Complement both sides} \quad (70)$$

$$\overline{M5} = \bar{\bar{a}} \wedge \bar{b} \wedge \bar{\bar{c}} \quad \text{DeMorgan 1} \quad (71)$$

$$\overline{M5} = a \wedge \bar{b} \wedge c \quad \text{Double negation} \quad (72)$$

$$\overline{M5} = m5 \quad (73)$$

Note that DeMorgan's also works with more than two inputs.

It turns out that converting a minterm to a maxterm works the same way. For example:

$$M5 = \bar{a} \vee b \vee \bar{c} \quad \text{Given} \quad (74)$$

$$m5 = a \wedge \bar{b} \wedge c \quad \text{Given} \quad (75)$$

$$\overline{m5} = \overline{a \wedge \bar{b} \wedge c} \quad \text{Complement both sides} \quad (76)$$

$$\overline{m5} = \bar{a} \vee \bar{\bar{b}} \vee \bar{c} \quad \text{DeMorgan 2} \quad (77)$$

$$\overline{m5} = \bar{a} \vee b \vee \bar{c} \quad \text{Double negation} \quad (78)$$

$$\overline{m5} = M5 \quad (79)$$

10.2 When to Apply the POS or SOP Form

If SOP and POS can be easily converted back and forth, it begs the question “Why concern ourselves with both forms?”

The answer lies in which of the two forms is easier to apply to a given situation. For example, the following truth table is best expressed in SOP form simply because there are fewer cases when the output is *true*.

<i>a</i>	<i>b</i>	<i>c</i>	<i>f</i>
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	0
1	1	1	1

Therefore there are fewer terms expressed in the SOP form describing it:

$$f = m3 \vee m5 \vee m7 \quad (80)$$

$$= (\bar{a} \wedge b \wedge c) \vee (a \wedge \bar{b} \wedge c) \vee (a \wedge b \wedge c) \quad (81)$$

... than there are in the POS form:

$$f = M0 \wedge M1 \wedge M2 \wedge M4 \wedge M6 \quad (82)$$

$$= (a \vee b \vee c) \wedge (a \vee b \vee \bar{c}) \wedge (a \vee \bar{b} \vee c) \wedge (\bar{a} \vee b \vee c) \wedge (\bar{a} \vee \bar{b} \vee c) \quad (83)$$

The shorter function may be more desirable to work with.

11 Simplification of SOP and POS Forms

While it is easy to express any Boolean function in SOP or POS form, it is desirable to be able to then reduce them to their simplest form.

To accomplish this, we apply the Boolean laws until the function becomes irreducible and has the fewest number of operators.

As an example we demonstrate a reduction of the following truth table using both the SOP and POS functions:

	a	b	c	f
0	0	0	0	0
1	0	0	1	0
2	0	1	0	1
3	0	1	1	0
4	1	0	0	1
5	1	0	1	1
6	1	1	0	1
7	1	1	1	1

SOP: (84)

$$f = m2 \vee m4 \vee m5 \vee m6 \vee m7 \quad \text{Given} \quad (85)$$

$$= (\bar{a} \wedge b \wedge \bar{c}) \vee (a \wedge \bar{b} \wedge \bar{c}) \vee (a \wedge \bar{b} \wedge c) \vee (a \wedge b \wedge \bar{c}) \vee (a \wedge b \wedge c) \quad \text{Expand the terms} \quad (86)$$

$$= (\bar{a} \wedge b \wedge \bar{c}) \vee (a \wedge \bar{b} \wedge (\bar{c} \vee c)) \vee (a \wedge b \wedge (\bar{c} \vee c)) \quad \text{Distributive of } \wedge \text{ over } \vee \quad (87)$$

$$= (\bar{a} \wedge b \wedge \bar{c}) \vee (a \wedge \bar{b} \wedge 1) \vee (a \wedge b \wedge 1) \quad \text{Complementation of } \vee \quad (88)$$

$$= (\bar{a} \wedge b \wedge \bar{c}) \vee (a \wedge \bar{b}) \vee (a \wedge b) \quad \text{Identity for } \wedge \quad (89)$$

$$= (\bar{a} \wedge b \wedge \bar{c}) \vee (a \wedge (\bar{b} \vee b)) \quad \text{Distributive of } \wedge \text{ over } \vee \quad (90)$$

$$= (\bar{a} \wedge b \wedge \bar{c}) \vee (a \wedge 1) \quad \text{Complementation of } \vee \quad (91)$$

$$= (\bar{a} \wedge b \wedge \bar{c}) \vee a \quad \text{Identity for } \wedge \quad (92)$$

$$= a \vee (\bar{a} \wedge b \wedge \bar{c}) \quad \text{Commutativity of } \vee \quad (93)$$

$$= a \vee (\bar{a} \wedge (b \wedge \bar{c})) \quad \text{Associativity of } \wedge \quad (94)$$

$$= (a \vee \bar{a}) \wedge (a \vee (b \wedge \bar{c})) \quad \text{Distributivity of } \vee \text{ over } \wedge \quad (95)$$

$$= 1 \wedge (a \vee (b \wedge \bar{c})) \quad \text{Complementation of } \vee \quad (96)$$

$$= a \vee (b \wedge \bar{c}) \quad \text{Identity for } \wedge \quad (97)$$

POS: (98)

$$f = M0 \wedge M1 \wedge M3 \quad \text{Given} \quad (99)$$

$$= (a \vee b \vee c) \wedge (a \vee b \vee \bar{c}) \wedge (a \vee \bar{b} \vee \bar{c}) \quad \text{Expand the terms} \quad (100)$$

$$= ((a \vee b) \vee (c \wedge \bar{c})) \wedge (a \vee \bar{b} \vee \bar{c}) \quad \text{Distributivity of } \vee \text{ over } \wedge \quad (101)$$

$$= ((a \vee b) \vee 0) \wedge (a \vee \bar{b} \vee \bar{c}) \quad \text{Complementation of } \wedge \quad (102)$$

$$= (a \vee b) \wedge (a \vee \bar{b} \vee \bar{c}) \quad \text{Identity for } \vee \quad (103)$$

$$= a \vee (b \wedge (\bar{b} \vee \bar{c})) \quad \text{Distributivity of } \vee \text{ over } \wedge \quad (104)$$

$$= a \vee ((b \wedge \bar{b}) \vee (b \wedge \bar{c})) \quad \text{Distributivity of } \wedge \text{ over } \vee \quad (105)$$

$$= a \vee (0 \vee (b \wedge \bar{c})) \quad \text{Complementation of } \wedge \quad (106)$$

$$= a \vee (b \wedge \bar{c}) \quad \text{Identity for } \vee \quad (107)$$

$$(108)$$

Ultimately, we see that both methods arrived at the same conclusion!